MuniVox · Modelo de vistas 4+1 (Kruchten)

Cuatro vistas arquitectónicas, cada una para un grupo de interesados; los escenarios (+1) las integran.

Vista Lógica

Interesados: usuarios finales, analistas.

Responde: ¿qué información y qué funciones ofrece el sistema?
Elementos: Ciudadano, Reporte (ticket), Categoría, Departamento, Operador, Evidencia, HistorialEstado y enumeraciones.

Diagrama: clases UML (esquema Prisma, sección 5.5).

Vista de Desarrollo

Interesados: equipo de desarrollo, mantenedores.

Responde: ¿cómo está organizado el código fuente?
Elementos: 5 proyectos (webhook-service, worker-service, socket-service, lambda-rest-api, munivox-dashboard) con sus módulos y paquetes.

Diagrama: paquetes / módulos.

Escenarios (+1)

Casos de uso que recorren y validan las cuatro vistas (ver vista de escenarios)

Vista de Procesos

Interesados: integradores, rendimiento.

Responde: ¿cómo se comunican las partes en tiempo de ejecución?
Elementos: cola BullMQ, escudo anti-duplicados, IA, Pub/Sub de Redis, WebSocket, Lambda y notificaciones.

Diagrama: secuencia (del reporte al aviso de cierre).

Vista Física

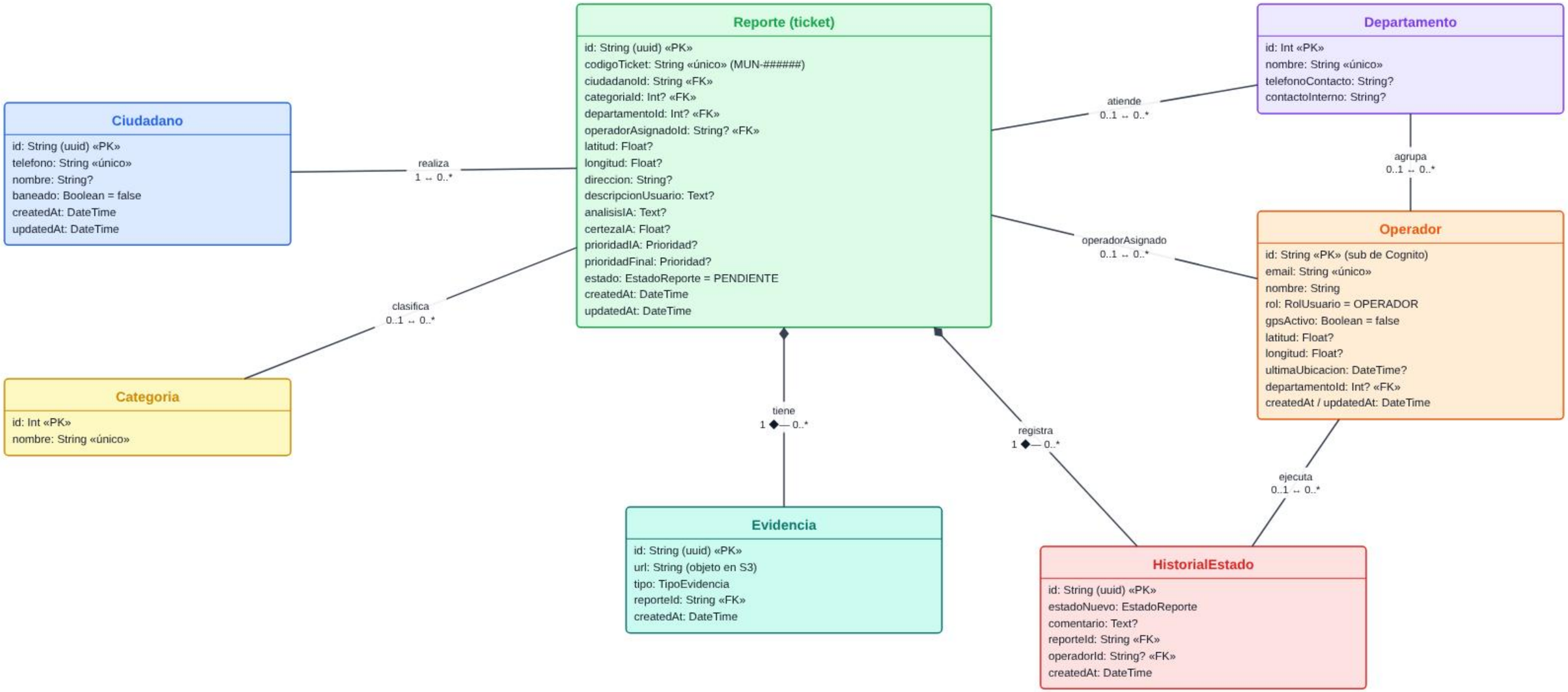
Interesados: ingeniería de sistemas, operación.

Responde: ¿en qué infraestructura se ejecuta cada componente?
Elementos: Azure Container Apps (3 servicios), AWS (Amplify, API Gateway + Lambda, ElastiCache, RDS, S3, Cognito) y servicios externos.

Diagrama: despliegue.

Vista Lógica · Modelo de dominio (clases UML)

Basado en schema.prisma (sección 5.5). ◆ = composición (borrado en cascada); « » = estereotipo.



Enumeraciones

«enumeration» EstadoReporte
- PENDIENTE
- EN_REVISION
- DESPACHADO
- RESUELTO
- DESCARTADO

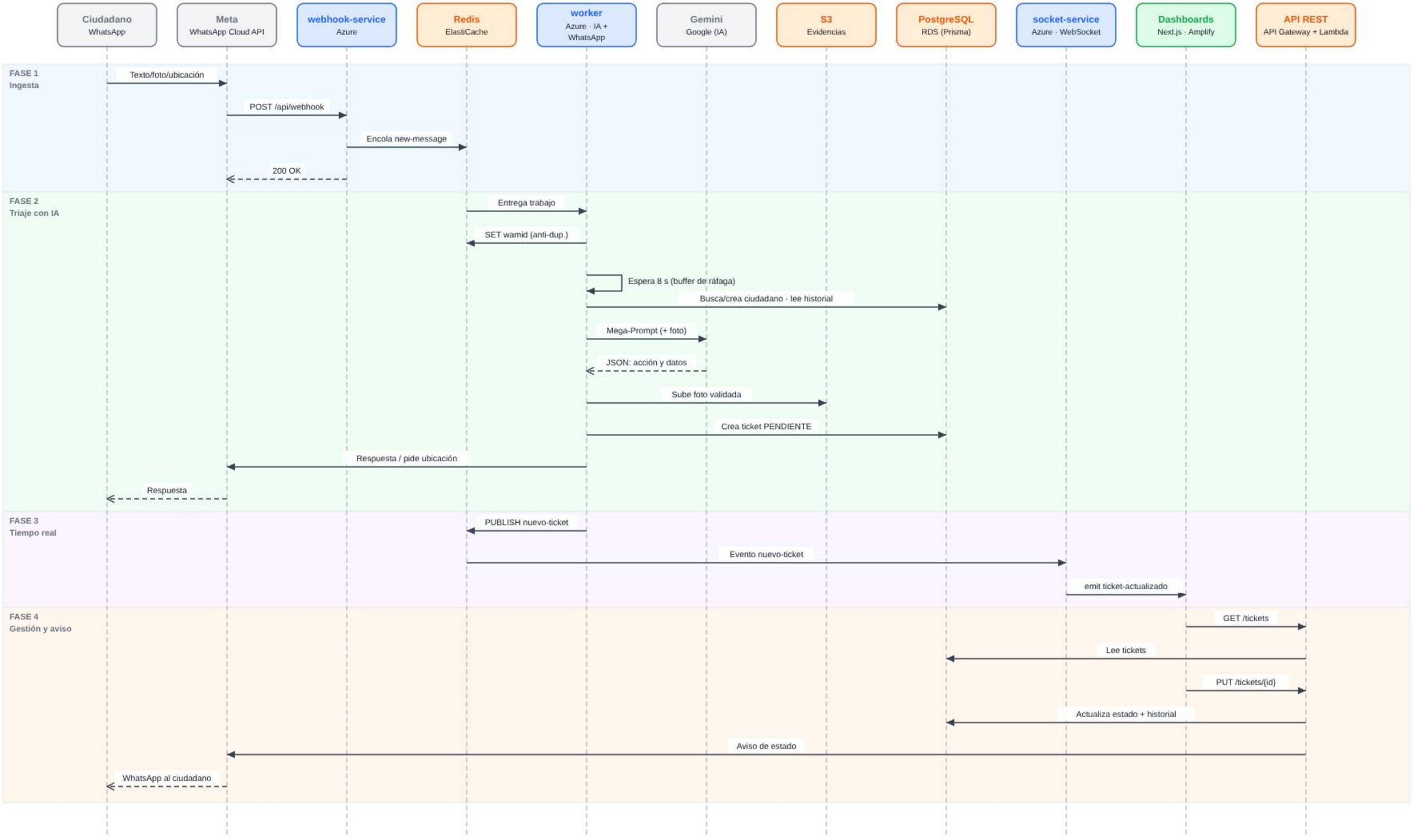
«enumeration» Prioridad
- CRITICA
- ALTA
- MEDIA
- BAJA

«enumeration» RolUsuario
- ADMINISTRADOR
- OPERADOR
- TERRENO

«enumeration» TipoEvidencia
- IMAGEN
- AUDIO
- VIDEO

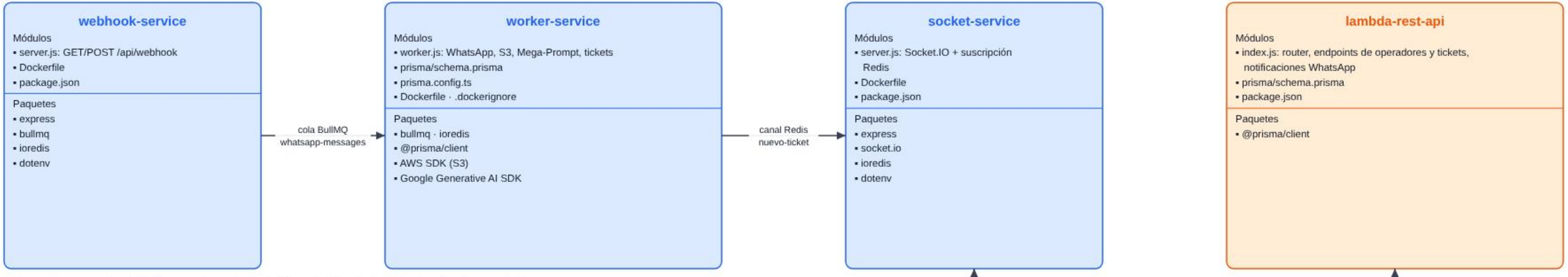
Vista de Procesos · Del reporte ciudadano al aviso de cierre

Diagrama de secuencia. Línea continua = solicitud/evento; punteada = respuesta. Asíncrono vía cola BullMQ y Pub/Sub de Redis.



Vista de Desarrollo · Organización del código

Cinco proyectos independientes. Los servicios se acoplan solo por contratos (cola, canal Redis, API REST), no por código compartido.



schema.prisma se mantiene idéntico en worker-service y lambda-rest-api (ambos trabajan sobre las mismas tablas).

Socket.IO (WebSocket)

API REST (HTTP/JSON)

munivox-dashboard (Next.js + TypeScript · AWS Amplify)

app/ (App Router)
- page.tsx → Dashboard Operador C4 (/)
- departamentos/page.tsx → Dashboard Departamentos
- terreno/page.tsx → App de Terreno (PWA)
- login/page.tsx
- api/auth/[...nextauth]/route.ts
- layout.tsx · Providers.tsx · globals.css

components/
- Map.tsx → motor geoespacial (Mapbox GL)
- Kanban.tsx
- Estadisticas.tsx
- AdminPanel.tsx

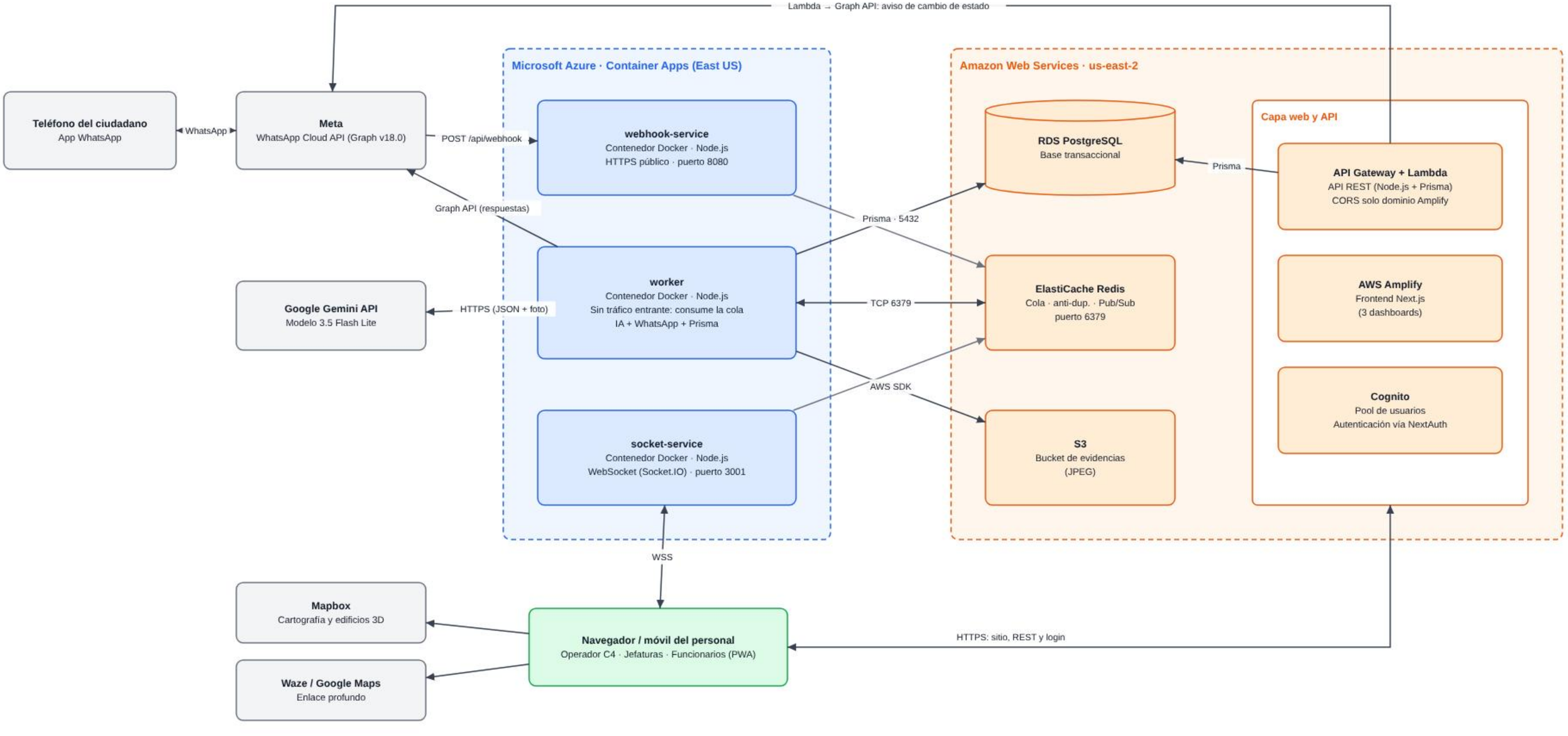
data/ · public/
- data/buin-boundary.json (contorno comunal)
- data/unidades-vecinales/ (19 GeoJSON)
- public/marcadores/ (iconos 2.5D .png/.gif)

Configuración y paquetes
- middleware.ts (control de acceso por sesión)
- amplify.yml · next.config.ts · tsconfig.json
- next · react · typescript
- mapbox-gl · socket.io-client
- next-auth · react-icons

Variables de entorno del frontend: NEXT_PUBLIC_API_GATEWAY_URL, NEXT_PUBLIC_MAPBOX_TOKEN y configuración de NextAuth/Cognito.
Los datos nunca se leen de la base directamente: siempre pasan por la API REST.

Vista Física · Despliegue

Servicios de ingesta y tiempo real en Azure Container Apps; datos, API, identidad y frontend en AWS (us-east-2).



Vista de Escenarios (+1) · Trazabilidad escenario → vistas

Cada escenario recorre elementos concretos de las cuatro vistas; si alguno no se puede explicar, falta algo en la arquitectura.

Escenario	Actores	Lógica	Procesos	Desarrollo	Física
E1 · Reportar una incidencia por WhatsApp	Ciudadano · IA (Gemini)	Ciudadano, Reporte, Evidencia, Categoría	Secuencia fases 1–2: webhook → cola → worker → IA → ticket	webhook-service, worker-service	Meta → Azure → Redis / RDS / S3 (AWS)
E2 · Completar ubicación o foto faltante	Ciudadano	Reporte (latitud/longitud, evidencias) en PENDIENTE	Acción CONTINUAR_TICKET; solicitud interactiva de ubicación	worker-service (worker.js)	Meta ↔ worker; RDS; S3
E3 · Emergencia vital (protocolo ABC)	Ciudadano · IA (Gemini)	Prioridad CRÍTICA; no se crea ticket	Acción DERIVACION_ABC: se informa 131 / 132 / 133	worker.js (Mega-Prompt, regla 4)	Gemini API → worker → Meta
E4 · Monitoreo en tiempo real en el C4	Operador C4	Reporte, Categoría, Operador (GPS)	Pub/Sub nuevo-ticket → WebSocket → recarga	socket-service, Map.tsx, app/page.tsx	Redis → socket-service (WSS) → navegador
E5 · Despacho y resolución en terreno	Jefatura · Funcionario	Estados, HistorialEstado, operadorAsignado	Asignar → aceptar/rechazar → navegar → resolver (PUT /tickets/{id})	departamentos/page.tsx, terreno/page.tsx, lambda index.js	Amplify → API Gateway/Lambda → RDS
E6 · Avisar al ciudadano cada cambio de estado	Sistema (Lambda)	Estados EN_REVISION, DESPACHADO, RESUELTO	Lambda → Graph API → WhatsApp del ciudadano	lambda-rest-api (módulo de notificaciones)	Lambda (AWS) → Meta → WhatsApp

Los escenarios E1–E6 resumen los casos de uso del sistema (el detalle de actores y relaciones está en el diagrama de casos de uso).